BACKEND ARCHITECTURE & DEFENSE MASTER GUIDE

คู่มือสถาปัตยกรรมระบบหลังบ้านและแนวข้อสอบป้องกันโปรเจกต์

ระบบบริหารจัดการสหกรณ์รับซื้อน้ำยางพารา (Clover Project)

เอกสารฉบับสมบูรณ์ครอบคลุมทุกมิติทางวิศวกรรมซอฟต์แวร์ของระบบหลังบ้าน (Backend Engine): การออกแบบสถาปัตยกรรมระดับองค์กร, ไปป์ไลน์การประมวลผลคำขอ, ระบบความปลอดภัยแบบแยกโดเมน (Domain-Separated HMAC), การบูรณาการ PostgreSQL และ Prisma ORM, ธุรกรรมการเงินแบบ ACID และแนวข้อสอบป้องกันโปรเจกต์เชิงลึก 25 ข้อพร้อมเฉลย

สถาปัตยกรรม: Layered Architecture (Routes → DAL → Prisma ORM → PostgreSQL)
สแต็กเทคโนโลยี: Node.js (ES Modules), Express 5.2.1, PostgreSQL, Prisma ORM 6.19.3, ExcelJS
การรักษาความปลอดภัย: Cryptographic HMAC-SHA256 Signed Cookies, Scrypt Hashing, RBAC, Zero-Trust
จัดทำโดย: คณะผู้พัฒนาโครงการ Clover Project • พฤศจิกายน 2569

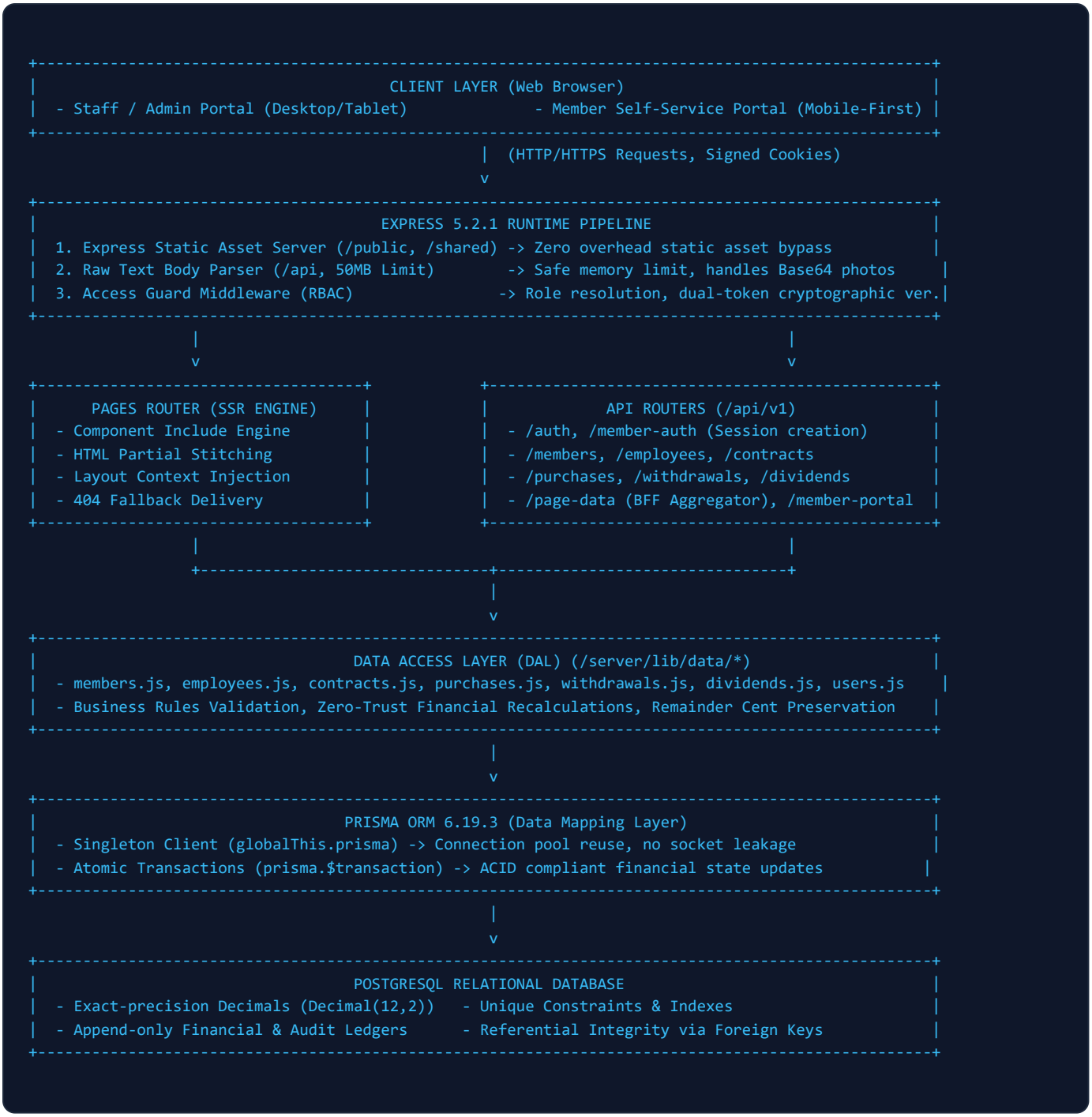
สารบัญเนื้อหา (Table of Contents)

ส่วนที่ 1.1 ภาพรวมสถาปัตยกรรมและเทคโนโลยีสแตก
ส่วนที่ 1.2 สภาพแวดล้อมและการเริ่มต้นระบบ (server.js, env.js)
ส่วนที่ 1.3 เลเยอร์ฐานข้อมูลและ Prisma ORM
ส่วนที่ 1.4 ระบบความปลอดภัยและ Dual-Session HMAC
ส่วนที่ 1.5 การควบคุมสิทธิ์ (RBAC) และ Access Guard
ส่วนที่ 1.6 การจัดการคำขอ ตัวช่วย HTTP และ Audit Log
ส่วนที่ 1.7 Data Access Layer และตรรกะทางธุรกิจ
ส่วนที่ 1.8 สถาปัตยกรรม REST API และ BFF Pattern
ส่วนที่ 1.9 การเรนเดอร์หน้าเว็บ (SSR) และโมดูลช่วยเสริม
ส่วนที่ 2.1 ข้อสอบหมวดสถาปัตยกรรมและหลักการออกแบบ [Q1-Q5]
ส่วนที่ 2.2 ข้อสอบหมวดความปลอดภัยและการยืนยันตัวตน [Q6-Q10]
ส่วนที่ 2.3 ข้อสอบหมวดธุรกรรมการเงินและ Concurrency [Q11-Q15]
ส่วนที่ 2.4 ข้อสอบหมวดฐานข้อมูลและ Prisma Optimization [Q16-Q20]
ส่วนที่ 2.5 ข้อสอบหมวดประสิทธิภาพและความเสถียร [Q21-Q25]

ส่วนที่ 1: เอกสารอธิบายการทำงานของ Backend ทั้งหมดอย่างละเอียด

1.1 ภาพรวมสถาปัตยกรรมและเทคโนโลยีสแตก (Architecture Overview)

ระบบหลังบ้านของ **Clover Project** ถูกสร้างขึ้นด้วยแนวคิด *Layered Architecture (สถาปัตยกรรมแบบแบ่งชั้น)* โดยใช้เทคโนโลยีระดับมาตรฐานองค์กร มุ่งเน้นความน่าเชื่อถือของข้อมูลการเงิน (Financial Data Integrity), ความปลอดภัยระดับสูง (Zero-Trust Security), และความเร็วในการตอบสนองคำขอ โดยมีองค์ประกอบหลักดังนี้:


CLIENT LAYER (Web Browser)
- Staff / Admin Portal (Desktop/Tablet)
- Member Self-Service Portal (Mobile-First)
(HTTP/HTTPS Requests, Signed Cookies)
EXPRESS 5.2.1 RUNTIME PIPELINE
1. Express Static Asset Server (/public, /shared) -> Zero overhead static asset bypass
2. Raw Text Body Parser (/api, 50MB Limit) -> Safe memory limit, handles Base64 photos
3. Access Guard Middleware (RBAC) -> Role resolution, dual-token cryptographic ver.
PAGES ROUTER (SSR ENGINE)
- Component Include Engine
- HTML Partial Stitching
- Layout Context Injection
- 404 Fallback Delivery
API ROUTERS (/api/v1)
- /auth, /member-auth (Session creation)
- /members, /employees, /contracts
- /purchases, /withdrawals, /dividends
- /page-data (BFF Aggregator), /member-portal
DATA ACCESS LAYER (DAL) (/server/lib/data/*)
- members.js, employees.js, contracts.js, purchases.js, withdrawals.js, dividends.js, users.js
- Business Rules Validation, Zero-Trust Financial Recalculations, Remainder Cent Preservation
PRISMA ORM 6.19.3 (Data Mapping Layer)
- Singleton Client (globalThis.prisma) -> Connection pool reuse, no socket leakage
- Atomic Transactions (prisma.$transaction) -> ACID compliant financial state updates
POSTGRESQL RELATIONAL DATABASE
- Exact-precision Decimals (Decimal(12,2))
- Unique Constraints & Indexes
- Append-only Financial & Audit Ledgers
- Referential Integrity via Foreign Keys

เทคโนโลยี / โมดูล	เวอร์ชัน	หน้าที่และความสำคัญในเชิงวิศวกรรม
Node.js (ES Modules)	v20+ LTS	รันไทม์ฝั่งเซิร์ฟเวอร์แบบ Non-blocking I/O ใช้ ES Modules แท้ (`type: "module"`) ทำให้โค้ดสะอาด เป็นโมดูลาร์ และทันสมัย
Express Framework	5.2.1	เว็บเซิร์ฟเวอร์ตัวหลัก จัดการ Routing, Middleware Pipeline และ HTTP Lifecycle ได้อย่างแม่นยำและมีประสิทธิภาพสูง
PostgreSQL	Relational DB	ฐานข้อมูลเชิงสัมพันธ์ระดับองค์กร รองรับ ACID Transactions และการคำนวณทศนิยมที่แม่นยำสูง (Numeric/Decimal) ปราศจาก Floating Point Inaccuracy
Prisma ORM	6.19.3	ตัวจัดการการเชื่อมต่อและจับคู่ข้อมูล (Object-Relational Mapping) ที่มี Type-Safety ป้องกัน SQL Injection โดยสมบูรณ์
ExcelJS	4.4.0	โมดูลสร้างและจัดรูปแบบไฟล์สเปรดชีต Excel (.xlsx) คุณภาพสูง สำหรับระบบรายงานสรุปยอดการรับซื้อน้ำยางพารา

1.2 สภาพแวดล้อมและการเริ่มต้นระบบ (Environment Configuration & Server Pipeline)

เซิร์ฟเวอร์เริ่มต้นการทำงานที่ไฟล์ `server/server.js` โดยมีลำดับการประมวลผล (Execution Pipeline) ที่ถูกออกแบบมาเพื่อความปลอดภัยและประสิทธิภาพสูงสุด:

ลำดับการทำงานของ Middleware Pipeline ใน server.js

1. **การโหลดตัวแปรสภาพแวดล้อม (`import "./env.js"`):** ถูกเรียกเป็นบรรทัดแรกสุด เพื่อให้แน่ใจว่าค่า `NODE_ENV` , `DATABASE_URL` , `SESSION_SECRET` ถูกอ่านเข้าสู่ `process.env` ก่อนที่โมดูลอื่นใดจะเริ่มทำงาน
2. **การพรางข้อมูลเซิร์ฟเวอร์ (`app.disable("x-powered-by")`):** ปิดการส่ง HTTP Header `X-Powered-By: Express` ป้องกันการถูกผู้ไม่หวังดีสแกนหาช่องโหว่ประจำเวอร์ชันของเซิร์ฟเวอร์ (Information Disclosure Mitigation)
3. **การส่งไฟล์สแตติกคงที่ (Static Assets Bypass):** เส้นทาง `/public` และ `/shared` ให้บริการผ่าน `express.static` โดยตั้งค่า `index: false` เพื่อป้องกัน Directory Browsing และไม่มีการตรวจสอบสิทธิ์ ทำให้เบราว์เซอร์ดาวน์โหลดไฟล์ CSS/JS/รูปภาพ ได้รวดเร็วที่สุด
4. **การรับข้อมูลเนื้อหา API (Raw Text Body Parser):** ใช้ `express.text({ type: () => true, limit: "50mb" })` สำหรับเส้นทาง `/api` รองรับการส่งภาพถ่ายรูปโปรไฟล์สมาชิกในรูปแบบ Base64 ขนาดใหญ่ โดยยังไม่ Parse เป็น JSON ทันทีเพื่อประหยัด CPU จะ Parse เฉพาะเมื่อ Endpoint นั้นต้องการ
5. **ตัวควบคุมการเข้าถึงส่วนกลาง (`app.use(accessGuard)`):** ด่านตรวจสิทธิ์คัดกรองทุกคำขอ แยกเส้นทางสาธารณะ, สมาชิก, พนักงาน และผู้ดูแลระบบ ก่อนส่งต่อไปยัง Router ปลายทาง
6. **เราเตอร์ API และหน้าเว็บ:** แมป `/api` ไปยัง `apiRouter` และหน้าเว็บทั่วไปไปยัง `pagesRouter`
7. **ตัวจัดการ 404 และ Error Handler รวมศูนย์:** ดักจับกรณีไม่พบหน้าด้วย `sendNotFound` และกรณีเกิดข้อผิดพลาดรุนแรงด้วย Global Error Middleware ตอบกลับเป็น JSON ในเส้นทาง API หรือข้อความมาตรฐานในหน้าเว็บ

```
// โค้ดสำคัญใน server/server.js
import "./env.js";
import express from "express";
import { accessGuard } from "./middleware/access-guard.js";
import { apiRouter } from "./routes/api.routes.js";
import { pagesRouter, sendNotFound } from "./routes/pages.routes.js";

const app = express();
app.disable("x-powered-by");

// 1. Static Assets (Public & Shared)
app.use(express.static(path.join(projectRoot, "public"), { index: false }));
app.use("/shared", express.static(path.join(projectRoot, "shared"), { index: false }));

// 2. High-Capacity Text Parser สำหรับ Base64 Photo Uploads
app.use("/api", express.text({ type: () => true, limit: "50mb" }));

// 3. Access Guard RBAC
app.use(accessGuard);

// 4. Mount API & Page Routers
app.use("/api", apiRouter);
app.use(pagesRouter);

// 5. Fallback 404 & Centralized Error Handler
app.use((req, res, next) => { sendNotFound(res).catch(next); });
app.use((error, req, res, next) => {
  console.error(error);
  if (res.headersSent) return next(error);
  const status = error.status || error.statusCode || 500;
  if (req.path.startsWith("/api")) {
    return res.status(status).json({ error: error.message || "Internal server error" });
  }
  return res.status(status).type("text").send("Internal Server Error");
});
```

1.3 เลเยอร์ฐานข้อมูลและ Prisma ORM (Database Schema & Prisma Layer)

ฐานข้อมูลถูกออกแบบตามหลัก 3rd Normal Form (3NF) เพื่อขจัดความซ้ำซ้อนของข้อมูล และใช้ชนิดข้อมูล `Decimal` สำหรับตัวเลขทางการเงินทั้งหมด เพื่อป้องกันปัญหาเศษสตางค์คลาดเคลื่อนจากมาตรฐาน IEEE 754 Floating Point ใน JavaScript

ความสัมพันธ์และโครงสร้างของเอนทิตีหลักใน prisma/schema.prisma

โมเดล (Model)	คีย์หลักและฟิลด์เด่น	หน้าที่และความสัมพันธ์ (Relationships)
`Member`	`id (cuid)`, `memberCode`, `walletBalance`, `dividendBalance`	สมาชิกเจ้าของสวนยาง มีกระเป๋าเงินสะสม 2 ส่วน: ยอดเงินขายยาง (walletBalance) และเงินปันผล (dividendBalance) มีความสัมพันธ์ One-to-Many กับ Purchase, Withdrawal, DividendPayment, MePair
`Employee`	`id (cuid)`, `employeeCode`, `status`	คนกรีดยาง / ลูกจ้างรับจ้างกรีดน้ำยาง มีความสัมพันธ์กับสัญญาคู่กรีด `MePair`
`MePair`	`pairCode`, `memberShare`, `employeeShare`	สัญญาจับคู่ระหว่าง Member กับ Employee เพื่อกำหนดสัดส่วนแบ่งรายได้ (เช่น 60/40, 50/50) มี Foreign Keys ชี้ไปยัง Member และ Employee
`Purchase`	`purchaseCode`, `rawWeightKg`, `dryPercentage`, `dryWeightKg`, `totalAmount`	ตารางบันทึกการรับซื้อน้ำยางพารา เก็บข้อมูลน้ำหนักสด เปอร์เซ็นต์ DRC น้ำหนักยางแห้ง ราคากลาง และการคำนวณเงินส่วนลูกจ้าง (employeePayout) และเจ้าของสวน (ownerPayout)
`Withdrawal`	`withdrawalCode`, `amount`, `balanceBefore`, `balanceAfter`	ประวัติการเบิกเงินสดของสมาชิก เป็นตารางแบบ Append-only บันทึกภาพถ่ายยอดเงินก่อนและหลังเบิก (Balance Snapshot) เพื่อการตรวจสอบย้อนหลังที่แม่นยำ
`DividendPayment`	`dividendCode`, `buddhistYear`, `rate`, `amount`, `balanceBefore`	ประวัติการจ่ายเงินปันผลประจำปี มีเงื่อนไข `@@unique([memberId, buddhistYear])` ป้องกันการจ่ายเงินปันผลซ้ำซ้อนให้สมาชิกในปีเดียวกันระดับฐานข้อมูล
`Staff`	`staffCode`, `username`, `password`, `role (ADMIN/STAFF)`	บัญชีผู้ใช้งานระบบสำหรับเจ้าหน้าที่และผู้ดูแลระบบ
`ActivityLog`	`logId`, `action`, `targetType`, `targetId`, `status`	ตาราง Audit Trail บันทึกทุกความเคลื่อนไหวในระบบ (Login, Create, Update, Delete)
`ReferencePrice` / `History`	`date`, `price`	ราคากลางอ้างอิงยางพาราประจำวัน พร้อมตาราง History บันทึกประวัติการเปลี่ยนแปลงราคา

Prisma Singleton Pattern (server/lib/prisma.js): ในสภาวะการพัฒนา (Development Mode) โค้ดจะถูก Reload บ่อยครั้ง หากสร้าง `new PrismaClient()` ทุกครั้งจะทำให้ Connection Pool ของ PostgreSQL เต็ม (Connection Exhaustion) ระบบจึงแก้ปัญหานี้ด้วยการเก็บ Client ไว้ใน `globalThis.prisma` เพื่อนำ Connection เดิมกลับมาใช้ซ้ำเสมอ

1.4 ระบบความปลอดภัยและการจัดการเซสชันแบบแยกโดเมน (Security & Dual-Session Management)

ระบบเลือกใช้สถาปัตยกรรมความปลอดภัยแบบ **Cryptographic Signed Cookies (HMAC-SHA256)** โดยไม่พึ่งพา Session Table ในฐานข้อมูล (Stateless Tokenized Cookie) ช่วยลดภาระการ Query ฐานข้อมูลในทุกๆ Request:

หลักการทำงานของ Cryptographic Domain Separation

ระบบแยกเซสชันของ **เจ้าหน้าที่ (Staff)** และ **สมาชิก (Member)** ออกจากกันโดยเด็ดขาด แม้ทั้งสองฝั่งจะอยู่บนเซิร์ฟเวอร์เดียวกัน โดยใช้เทคนิคการกระจายคีย์ทางคณิตศาสตร์ (Key Derivation):

- **Staff Session Cookie (`session`):** เซ็นด้วยรหัสลับหลัก `process.env.SESSION_SECRET` เก็บข้อมูล `{ staffId, username, role, exp }`
- **Member Session Cookie (`member_session`):** สร้างกุญแจย่อยเฉพาะทางขึ้นมาใหม่ด้วยคำสั่ง:

```
crypto.createHmac("sha256", secret).update("member-session-v1").digest();
```

เก็บข้อมูล `{ type: "member", memberId, memberCode, exp }`

> **ผลลัพธ์ทางความปลอดภัย:** แม้ผู้ไม่หวังดีจะคัดลอก Cookie ของสมาชิกแล้วเปลี่ยนชื่อเป็น Cookie ของเจ้าหน้าที่ ลายเซ็น HMAC ก็จะไม่มีทางตรงกัน เพราะกุญแจที่ใช้ตรวจสอบเป็นคนละตัวกันทางคณิตศาสตร์ จึงป้องกันการเลื่อนระดับสิทธิ์ (Privilege Escalation) ได้ 100%

ความปลอดภัยของรหัสผ่าน (server/lib/auth.js)

รหัสผ่านของระบบไม่ได้ใช้ MD5 หรือ SHA-256 ธรรมดา แต่ใช้ฟังก์ชัน `crypto.scrypt` ซึ่งเป็นอัลกอริทึมที่ออกแบบมาให้ใช้หน่วยความจำสูง (Memory-Hard Function) เพื่อต้านทานการถอดรหัสด้วยฮาร์ดแวร์ขุดเหมือง GPU/ASIC:

- สร้าง Salt แบบสุ่มขนาด 16 ไบต์ใหม่ทุกครั้ง (`crypto.randomBytes(16)`) ป้องกัน Rainbow Table Attack
- คำนวณคีย์ความยาว 64 ไบต์ และเก็บในรูปแบบ `<salt>:<hash>`
- ตรวจสอบรหัสผ่านด้วย `crypto.timingSafeEqual` : เปรียบเทียบระดับบิตโดยใช้เวลาคงที่เสมอ ป้องกันการโจมตีประเภท *Timing Attack* (การจับเวลาการทำงานของ CPU เพื่อเดาตัวอักษรของรหัสผ่าน)

1.5 การควบคุมสิทธิ์ (Role-Based Access Control) และ Access Guard

`server/middleware/access-guard.js` ทำหน้าที่เป็นเกราะป้องกันหน้าเว็บและ API ทั้งหมด โดยกำหนดสิทธิ์ตามบทบาท (RBAC):

กลุ่มผู้ใช้ / เส้นทาง	สิทธิ์ที่ต้องใช้	พฤติกรรมเมื่อไม่ได้รับอนุญาต
หน้าและ API สาธารณะ `/api/auth/*` , `/api/member-auth/*`	ไม่ต้องยืนยันตัวตน (Public)	อนุญาตให้ผ่านได้ทันที
ระบบจัดการผู้ใช้งานและ Audit Log `/users` , `/audit-log` , `/api/users/*`	ต้องเป็น Staff ที่มี Role = `ADMIN` เท่านั้น	- หากยังไม่ล็อกอิน: Redirect ไป `/login?next=...` - หากเป็น STAFF ทั่วไป: API ตอบ 403 Forbidden, หน้าเว็บ Redirect ไปหน้าแรก
งานปฏิบัติการประจำวัน `/purchases` , `/withdrawals` , `/dividends`	Staff Role = `STAFF` หรือ `ADMIN`	API ตอบ 401 Unauthorized, หน้าเว็บ Redirect ไป `/login`
พอร์ทัลสมาชิก `/member/*` , `/api/member-portal/*`	ต้องมี `member_session` ที่ถูกต้อง	API ตอบ 401 Unauthorized พร้อม URL ไป `/member/login`

1.6 การจัดการคำขอ ตัวช่วย HTTP และระบบบันทึกประวัติ (Audit Trail)

การจัดการการส่งข้อมูลและการตอบสนองถูกสร้างเป็นเลเยอร์ส่วนกลางใน `server/lib/` :
การแยกวิเคราะห์ JSON อย่างปลอดภัย (`server/lib/request-body.js`): ฟังก์ชัน `readJsonBody(req)` ป้องกันข้อผิดพลาด JSON Syntax Error ไม่ให้ทำให้โปรเซสเซิร์ฟเวอร์ล่ม (Crash) โดยจะส่ง `ApiError.badRequest("รูปแบบข้อมูล JSON ไม่ถูกต้อง")` กลับไปอย่างสุภาพ
การจัดโครงสร้างความผิดพลาด (`server/lib/api-error.js`): คลาส `ApiError` สืบทอดจาก `Error` โดยกำหนด HTTP Status Code อย่างเป็นระบบ (400, 401, 403, 404, 409, 500)
การตรวจสอบข้อมูลนำเข้า (`server/lib/validate.js`): ตรวจสอบความยาวของเลขประจำตัวประชาชน 13 หลัก, หมายเลขโทรศัพท์ 10 หลัก และรูปแบบรหัสสมาชิก
ระบบบันทึกประวัติการใช้งาน (`server/lib/activity-log.js`): บันทึกการกระทำสำคัญ เช่น การล็อกอินสำเร็จ/ล้มเหลว, การบันทึกรับซื้อ, การเบิกเงิน, การจ่ายปันผล และการสร้าง/แก้ไขสัญญาจ้าง ลงในตาราง `ActivityLog` โดยอัตโนมัติ เพื่อรองรับการตรวจสอบทุจริตย้อนหลัง

1.7 Data Access Layer และตรรกะทางธุรกิจ (Business Logic & Transactions)

เลเยอร์การเข้าถึงข้อมูลใน `server/lib/data/*` แยกการคำนวณและกฎธุรกิจออกจาก Route Handlers อย่างเด็ดขาด:

1. ตรรกะการรับซื้อน้ำยางพารา (server/lib/data/purchases.js)

ใช้หลักการ **Zero-Trust Calculation** โดยเซิร์ฟเวอร์จะนำ `rawWeightKg` , `dryPercentage` และราคากลางมาคำนวณใหม่ทั้งหมด:

```
// สูตรการคำนวณใน createPurchase
const dryWeightKg = (input.rawWeightKg * input.dryPercentage) / 100;
const totalAmount = dryWeightKg * input.marketPrice;

// ปัดเศษส่วนของคนกรีดยาง แล้วนำยอดรวมลบออก เพื่อรักษาเศษสตางค์ไม่ให้สูญหาย
const employeePayout = Math.round(totalAmount * (seller.employeeShare / 100) * 100) / 100;
const ownerPayout = Math.round((totalAmount - employeePayout) * 100) / 100;

// บันทึกด้วย ACID Transaction
const [purchase] = await prisma.$transaction([
  prisma.purchase.create({ data: { ... } }),
  prisma.member.update({
    where: { id: seller.memberId },
    data: { walletBalance: { increment: ownerPayout } }, // อัปเดตยอดสะสมอัตโนมัติ
  }),
]);
```

2. ตรรกะการเบิกเงินสะสม (server/lib/data/withdrawals.js)

ตรวจสอบว่ายอดเงินที่จะเบิกต้องมากกว่า 0 และไม่เกินยอดสะสมคงเหลือ (`walletBalance`) จากนั้นทำการบันทึกประวัติการเบิกและหักยอดเงินคงเหลือในคราวเดียวกันด้วย `prisma.$transaction` พร้อมเก็บ `balanceBefore` และ `balanceAfter` เป็นประวัติ snapshot แบบถาวร

3. ตรรกะการคำนวณและจ่ายเงินปันผลประจำปี (server/lib/data/dividends.js)

เซิร์ฟเวอร์จะรวมน้ำหนักเนื้อยางแห้งจริงของสมาชิกแต่ละคนจากรายการรับซื้อ (`Purchase`) ตลอดทั้งปี พ.ศ. จากนั้นคูณด้วยอัตราเงินปันผล (บาท/กก.) และทำการจ่ายใน Transaction เดียว:

- บันทึกแถวประวัติใน `DividendPayment`
- บวกเงินเข้า `walletBalance` (ให้สมาชิกเบิกเป็นเงินสดได้)
- บวกเงินเข้า `dividendBalance` (เก็บบันทึกยอดปันผลสะสมตลอดชีพของสมาชิก)
- มี Database Unique Constraint `@@unique([memberId, buddhistYear])` ล็อกไม่ให้เกิดการจ่ายซ้ำปีเดิมอย่างเด็ดขาด

1.8 สถาปัตยกรรม REST API และ BFF Pattern (Backend-For-Frontend)

โครงสร้าง Routing ใน `server/routes/api.routes.js` รวม Resource ต่างๆ เป็นระเบียบ พร้อมการประยุกต์ใช้แนวคิด **Backend-For-Frontend (BFF Pattern)** ใน `page-data.routes.js` :

> **ข้อได้เปรียบของการใช้ BFF Pattern (page-data.routes.js)**
> ในเว็บแอปพลิเคชันทั่วไป เมื่อเปิดหน้าจอ 1 หน้า (เช่น หน้ารับซื้อ หรือหน้าคำนวณปันผล) เบราว์เซอร์จะต้องยิง Request ไปยังเซิร์ฟเวอร์หลายครั้ง (Waterfall Network Requests) เช่น ขอข้อมูลสมาชิก, ขอราคากลาง, ขอยอดขายย้อนหลัง ทำให้หน้าเว็บโหลดช้าและกินแบนด์วิธสูง
>
> **วิธีแก้ของ Clover Project:** ออกแบบ Endpoint `/api/page-data/:pageName` ซึ่งทำหน้าที่รวบรวม (Aggregate) ข้อมูลทั้งหมดที่จำเป็นสำหรับการเรนเดอร์หน้านั้นๆ มาจากหลายโมเดลในฐานข้อมูล แล้วส่งกลับไปยังเบราว์เซอร์ใน **HTTP Request เดียว (Single Roundtrip)** ทำให้หน้าจอแสดงผลได้รวดเร็วทันที

1.9 การเรนเดอร์หน้าเว็บ (Server-Side Include) และโมดูลช่วยเสริม

ระบบรวมหน้าเว็บด้วย `@include` (`server/routes/pages.routes.js`): เซิร์ฟเวอร์นำไฟล์ HTML มาประกบกับส่วนประกอบย่อย (Partials) เช่น แถบเมนูด้านข้าง (Sidebar), แถบหัวเรื่อง (Header) และแทรกบริบทผู้ใช้ (User Context) ลงใน HTML ก่อนส่งให้เบราว์เซอร์ ทำให้ไม่ต้องใช้ Template Engine หนักๆ อย่าง EJS หรือ Pug
การสร้างไฟล์รายงาน Excel ด้วย ExcelJS (`server/lib/export/purchase-summary-workbook.js`): สร้างไฟล์รายงานการรับซื้อน้ำยางพาราประจำเดือนและประจำปี โดยมีการจัดความกว้างคอลัมน์, การใส่สูตรคำนวณอัตโนมัติ, การจัดสีหัวตาราง และการฟอร์แมตตัวเลขทศนิยมสองตำแหน่งในระดับ Binary Buffer พร้อมส่งให้ผู้ใช้ดาวน์โหลดได้ทันที
ฐานข้อมูลที่อยู่ไทยแบบฝังตัว (`server/lib/thai-address.js`): เก็บข้อมูลตำบล อำเภอ จังหวัด และรหัสไปรษณีย์ทั่วประเทศไทยไว้ในหน่วยความจำ (In-Memory Lookup) ทำให้การค้นหารหัสไปรษณีย์และเติมชื่อตำบล/อำเภออัตโนมัติเกิดขึ้นในระดับ Sub-millisecond (< 1ms)
โมดูลฟอร์แมตข้อมูลแบบใช้ร่วมกัน (`shared/format.js`): ฟังก์ชันแปลงวันที่เป็น พ.ศ. ภาษาไทย, การจัดรูปแบบสกุลเงินบาท (`formatCurrency`) ที่เขียนแบบ Isomorphic JavaScript สามารถรันได้ทั้งบน Node.js Backend และบนเบราว์เซอร์ Client โดยไม่มีความคลาดเคลื่อน

ส่วนที่ 2: รวมแนวข้อสอบป้องกันโปรเจกต์เชิงลึก (Backend Defense Q&A)

รวบรวม 25 คำถามเชิงลึกครอบคลุมทุกมิติทางเทคนิคที่คณะกรรมการสอบโครงงานนิยมถาม พร้อมเจตนาของคำถามและแนวทางการตอบอย่างมืออาชีพ:

หมวดที่ 1: สถาปัตยกรรมและหลักการออกแบบระบบ (System Design & Architecture)

Q1 ทำไมจึงเลือกใช้สถาปัตยกรรมแบบ Layered Architecture (Routes → DAL → Prisma → DB) แทนที่จะเขียนคำสั่งคิวรีฐานข้อมูลลงใน Route Controller โดยตรง?

> **เจตนาของผู้สอบ:** วัดความเข้าใจในหลักการออกแบบซอฟต์แวร์ (Separation of Concerns, Maintainability, Testability) และการป้องกันปัญหาโค้ดพันกัน (Spaghetti Code)

แนวทางการตอบ:

เหตุผลหลักคือการปฏิบัติตามหลักการ **Separation of Concerns (การแยกหน้าที่ความรับผิดชอบ)** เพื่อความยืดหยุ่นและการบำรุงรักษาในระยะยาว:

- **Routes Layer (`server/routes/`):** มีหน้าที่รับผิดชอบเฉพาะเรื่อง HTTP Protocol เท่านั้น เช่น การอ่าน Parameter, การตรวจสอบ Headers, การส่ง HTTP Status Code (200, 400, 404, 500) และการจัดการ Cookies
- **Data Access Layer - DAL (`server/lib/data/`):** เป็นหัวใจในการกักเก็บ *Business Logic* และกฎเกณฑ์ทางธุรกิจ เช่น สูตรการคำนวณน้ำหนักยางแห้ง DRC, การจัดสรรรายได้ระหว่างสมาชิกกับคนกรีดยาง, การตรวจสอบยอดเงินคงเหลือ และการเปิด ACID Transaction
- **Prisma ORM & DB Layer:** ดูแลเรื่องการเชื่อมต่อฐานข้อมูลและการทำ Object-Relational Mapping

หากในอนาคตต้องการเปลี่ยนระบบรับคำขอ เช่น มีการเพิ่มคิวงานเบื้องหลัง (Background Worker / Message Queue) หรือเพิ่มช่องทาง GraphQL หรือ CLI Tool เราจะสามารถเรียกใช้งานฟังก์ชันใน DAL ได้ทันทีโดยไม่ต้องเขียนตรรกะการคำนวณหรือคิวรีฐานข้อมูลซ้ำซ้อน และยังทำให้การเขียน Unit Test ทำได้ง่ายขึ้นเพราะสามารถทดสอบตรรกะทางธุรกิจได้โดยไม่ต้องจำลอง HTTP Request

Q2 เหตุใดระบบจึงออกแบบสถาปัตยกรรมแบบไฮบริด คือมีทั้ง Server-Side Rendering (SSR) ร่วมกับ BFF Aggregated Page Data API?

> **เจตนาของผู้สอบ:** ทดสอบความรู้เรื่องประสิทธิภาพเครือข่าย (Network Latency), Initial Page Load Time และความแตกต่างระหว่าง SSR กับ Client-Side Hydration

แนวทางการตอบ:

การออกแบบนี้เป็นการผสมผสานข้อดีของ **Multi-Page Application (MPA)** และ **Single-Page Dynamic Experience**:

- **การเรนเดอร์โครงสร้างหน้าเว็บด้วย SSR (`pages.routes.js`):** ช่วยให้เบราว์เซอร์ได้รับโครงสร้างหลักของหน้าเว็บ (HTML Layout, Navigation, Header, Stylesheet) มาพร้อมใช้งานทันทีตั้งแต่ First Paint โดยไม่ต้องรอให้ JavaScript ทำงานเสร็จก่อน ส่งผลให้ First Contentful Paint (FCP) รวดเร็วมาก
- **การดึงข้อมูลด้วย BFF Pattern (`page-data.routes.js`):** แก้ปัญหาคลาสสิกของ SPA ดั้งเดิมที่มักจะเกิด *Waterfall Requests* (เบราว์เซอร์ต้องยิง API ขอสมาชิก → รอผล → ยิงขอราคากลาง → รอผล → ยิงขอยอดขาย) ระบบ Clover ใช้ BFF รวบรวมข้อมูลทั้งหมดของหน้านั้นที่เซิร์ฟเวอร์ แล้วส่งกลับมาใน **HTTP Roundtrip เดียว** ลดภาระของเครือข่ายลงได้อย่างมหาศาล และทำให้ฝั่งหน้าบ้านเขียนโค้ดผูกข้อมูลกับ DOM ได้ง่ายดาย ไม่ซับซ้อน

Q3 ใน server.js มีการใช้ express.text({ type: () => true, limit: "50mb" }) แทน express.json() ทั่วไป เพราะเหตุใด และมีความเสี่ยงหรือผลกระทบอย่างไร?

> **เจตนาของผู้สอบ:** ตรวจสอบว่าผู้พัฒนาเข้าใจพฤติกรรมของ Middleware ใน Express อย่างแท้จริงหรือไม่ และคำนึงถึงเรื่อง Denial of Service (DoS) หรือไม่

แนวทางการตอบ:

สาเหตุที่ต้องใช้: ในระบบสหกรณ์ มีฟังก์ชันการลงทะเบียนสมาชิกและคนกรีดยาง ซึ่งจำเป็นต้องมีการแนบภาพถ่ายใบหน้าจริงและภาพถ่ายบัตรประชาชนเพื่อยืนยันตัวตน โดยส่งขึ้นมาในรูปแบบข้อความ Data URL Base64 หากใช้ `express.json({ limit: '1mb' })` แบบมาตรฐาน จะเกิดข้อผิดพลาด `PayloadTooLargeError: request entity too large` การใช้ `express.text()` พร้อมกำหนด Limit 50MB ช่วยให้เซิร์ฟเวอร์สามารถรับข้อมูลภาพถ่ายความละเอียดสูงได้โดยไม่เกิดข้อผิดพลาด

การบริหารจัดการความเสี่ยงและทรัพยากร:

1. **การประหยัด CPU:** การรับเป็น Raw Text จะยังไม่มีการแปลงสตริงเป็น JavaScript Object ในระดับ Global Middleware ทำให้ประหยัด Memory และ CPU รอบแรก โดยจะ Parse ด้วย `readJsonBody(req)` เฉพาะใน Route ที่จำเป็นต้องใช้ข้อมูลเท่านั้น
2. **การป้องกัน Memory Exhaustion (DoS):** มีการกำหนดเพดานขนาดสูงสุดไว้ชัดเจนที่ 50MB หากมีคำขอที่ขนาดเกิน ระบบจะตัดการทำงานทันที และมีการจำกัดการเข้าถึง API ส่วนใหญ่ผ่าน `accessGuard` เพื่อป้องกันไม่ให้บุคคลภายนอกที่ไม่ได้รับอนุญาตยิงข้อมูลขนาดใหญ่เข้ามาก่อกวนเซิร์ฟเวอร์

Q4 ระบบใช้ Singleton Pattern ใน server/lib/prisma.js เพื่อจุดประสงค์ใด หากไม่ทำเช่นนี้จะเกิดปัญหาอะไรกับ Node.js Event Loop และ Database Connections?

> **เจตนาของผู้สอบ:** วัดความเข้าใจเชิงลึกเรื่อง Connection Pooling, Resource Leakage และพฤติกรรมของโมดูลใน Node.js

แนวทางการตอบ:

ในการทำงานของ **PrismaClient** ตัวอินสแตนซ์จะสร้าง *Connection Pool* เพื่อเปิดช่องทางการสื่อสารผ่าน TCP Socket ไปยัง PostgreSQL (โดยทั่วไปประมาณ 10-20 connections ต่อ 1 client)

ในระหว่างการพัฒนาซอฟต์แวร์ เราใช้โหมด Development ซึ่งมีการแก้ไขไฟล์และรีโหลดโมดูลซ้ำๆ หากทุกครั้งที่มีการรีโหลด เซิร์ฟเวอร์ทำการสร้าง `new PrismaClient()` ใหม่ การเชื่อมต่อเดิมที่เปิดค้างอยู่จะไม่ถูกปิดในทันที จะเกิดสภาวะที่เรียกว่า **Connection Leak (การรั่วไหลของการเชื่อมต่อ)** จนในที่สุดจะเกินขีดจำกัด `max_connections` ของ PostgreSQL ทำให้ฐานข้อมูลปฏิเสธการเชื่อมต่อใหม่และเซิร์ฟเวอร์เกิด Crash ด้วยข้อผิดพลาด "*Can't reach database server*" หรือ "*Too many connections*"

การนำ `globalThis.prisma` มาใช้ เป็นการการันตีว่าทั้งกระบวนการรันไทม์จะมีอินสแตนซ์ของ Prisma Client เพียงตัวเดียวเสมอ (Singleton) ไม่ว่าจะมีการรีโหลดโค้ดกี่ครั้งก็ตาม ทำให้จำนวน Database Connection คงที่และปลอดภัยสูงสุด

Q5 การจัดการ State ของแอปพลิเคชันมีการเก็บข้อมูลไว้ในหน่วยความจำของเซิร์ฟเวอร์หรือไม่ และสถาปัตยกรรมนี้รองรับการขยายระบบแนวนอน (Horizontal Scaling) ได้อย่างไร?

> **เจตนาของผู้สอบ:** ประเมินความเข้าใจเรื่อง Stateless Architecture, Sticky Sessions และการออกแบบระบบให้พร้อม Scale ด้วย Cluster หรือ Load Balancer

แนวทางการตอบ:

ระบบหลังบ้านของ Clover Project ถูกออกแบบให้เป็น **Stateless Server (เซิร์ฟเวอร์ไร้สถานะ)** โดยสมบูรณ์ในแง่ของเซสชันผู้ใช้:

- **เซสชันผู้ใช้ (Session State):** ไม่มีการเก็บตัวแปร Session ไว้ใน RAM ของโหนดเซิร์ฟเวอร์ (In-Memory Session Store) แต่ใช้เทคนิค *Self-contained Signed Cookie (HMAC-SHA256)* ข้อมูลสถานะการยืนยันตัวตนถูกเข้ารหัสและลงนามดิจิทัลส่งกลับไปเก็บที่เบราว์เซอร์ของผู้ใช้
- **ข้อมูลธุรกิจ (Business State):** สถานะทางการเงิน กระเป๋าเงิน และประวัติทั้งหมด จัดเก็บไว้ที่ศูนย์กลางคือ PostgreSQL

ความสามารถในการขยายระบบแนวนอน (Horizontal Scaling): เมื่อต้องการขยายระบบเพื่อรองรับผู้ใช้เพิ่มขึ้น เราสามารถวางเซิร์ฟเวอร์ Node.js เพิ่มเป็น 5 หรือ 10 โหนดด้านหลัง Reverse Proxy / Load Balancer (เช่น Nginx หรือ AWS ALB) ได้ทันที โดยไม่ต้องใช้ Sticky Session (Session Affinity) เพราะทุกโหนดที่แชร์ `SESSION_SECRET` เดียวกัน จะสามารถอ่านและตรวจสอบความถูกต้องของ Cookie ได้เหมือนกันทุกประการ

หมวดที่ 2: ความปลอดภัย การยืนยันตัวตน และการควบคุมสิทธิ์ (Security, Authentication & RBAC)

Q6 ทำไมระบบจึงเลือกใช้ Signed Cookie (HMAC-SHA256) แทนการเก็บ JWT ใน LocalStorage หรือการเก็บ Session Table ในฐานข้อมูล?

> **เจตนาของผู้สอบ:** วัดความเข้าใจเรื่อง Web Security (XSS vs CSRF), Storage Mechanisms และ Performance Overhead ของ Database Lookups

แนวทางการตอบ:

การเปรียบเทียบในเชิงวิศวกรรมความปลอดภัยและประสิทธิภาพ:

- **ทำไมไม่เก็บ Token ใน LocalStorage:** LocalStorage ของเบราว์เซอร์สามารถถูกเข้าถึงได้โดยตรงผ่านคำสั่ง JavaScript ใดๆ ก็ตามบนหน้าเว็บ หากระบบถูกโจมตีด้วยช่องโหว่ Cross-Site Scripting (XSS) เช่น ผ่านสคริปต์ภายนอก ผู้โจมตีจะสามารถขโมย Token ออกไปได้อย่างง่ายดาย ระบบจึงเลือกใช้ **Cookie ที่มีแฟล็ก** `HttpOnly` ซึ่งเบราว์เซอร์จะไม่อนุญาตให้ JavaScript ใดๆ อ่านค่าคุกกี้นี้ได้โดยเด็ดขาด
- **ทำไมไม่ใช้ Session Table ในฐานข้อมูล:** หากเก็บ Session ในฐานข้อมูล ทุกๆ คำขอที่วิ่งเข้ามา ไม่ว่าจะขอหน้าเว็บหรือเรียก API เซิร์ฟเวอร์จะต้องยิงคิวรี `SELECT * FROM sessions WHERE id = ...` ไปยังฐานข้อมูลก่อนเสมอ ทำให้ฐานข้อมูลรับภาระหนักมากโดยไม่จำเป็น การใช้ Signed Cookie ช่วยให้ตรวจสอบความถูกต้องได้ในระดับ CPU ทันทีโดยไม่ต้องแตะฐานข้อมูล
- **การป้องกันการปลอมแปลง (Tamper Protection):** ตัวโทเค็นถูกผนึกด้วย HMAC-SHA256 หากผู้ใช้พยายามแอบเปลี่ยน `role: "STAFF"` เป็น `role: "ADMIN"` ใน Payload ลายเซ็นดิจิทัลจะไม่ตรงกับค่า Hash ใหม่ เซิร์ฟเวอร์จะปฏิเสธคำขอนั้นทันที

Q7 อธิบายหลักการทำงานของฟังก์ชัน Hash รหัสผ่านด้วย crypto.scrypt และทำไมจึงต้องใช้ crypto.timingSafeEqual ในการตรวจสอบรหัสผ่าน?

> **เจตนาของผู้สอบ:** ประเมินความรู้ด้านวิทยาการเข้ารหัสลับ (Applied Cryptography) และความเข้าใจในเรื่อง Side-Channel Attacks / Timing Attacks

แนวทางการตอบ:

1. การทำงานของ `crypto.scrypt` (server/lib/auth.js):
อัลกอริทึมทั่วไปอย่าง MD5 หรือ SHA-256 ถูกออกแบบมาให้ประมวลผลเร็ว จึงง่ายต่อการถูกแฮกเกอร์ใช้เครื่องคอมพิวเตอร์กราฟิก (GPU) ยิงคำนวณรหัสผ่านแบบ Brute-Force ได้หลายพันล้านครั้งต่อวินาที แต่ **Scrypt** เป็น *Memory-Hard Key Derivation Function* ที่ถูกออกแบบมาให้ต้องใช้ทรัพยากรหน่วยความจำ (RAM) จำนวนมากในทุกครั้งที่คำนวณ ทำให้การสร้างชิปพิเศษ (ASIC) หรือใช้ GPU ถอดรหัสทำได้ยากและมีต้นทุนมหาศาล ระบบยังสุ่ม **Salt ขนาด 16 ไบต์** ใหม่ทุกครั้ง ทำให้รหัสผ่านที่เหมือนกันสองบัญชี มีค่าแฮชที่บันทึกต่างกันโดยสิ้นเชิง ป้องกันการโจมตีด้วย Rainbow Tables

2. ความสำคัญของ `crypto.timingSafeEqual` :
หากเปรียบเทียบสตริงรหัสผ่านด้วยตัวดำเนินการปกติ `if (candidate === hash)` คอมไพเลอร์จะตรวจสอบทีละตัวอักษรจากซ้ายไปขวา และ **จะหยุดทำงานทันทีที่พบตัวอักษรที่ไม่ตรงกัน (Early Exit)** ผู้โจมตีสามารถวัดเวลาการตอบสนองของเซิร์ฟเวอร์ในระดับนาโนวินาที (Nanoseconds) เพื่อคาดเดาว่าตัวอักษรตัวแรก ตัวที่สอง ถูกต้องแล้วหรือไม่ (Timing Attack) การใช้ `timingSafeEqual` จะบังคับให้ CPU ใช้เวลาในการเปรียบเทียบข้อมูลทุกไบต์เท่ากันเสมอ ไม่ว่าจะตรงหรือไม่ตรง จึงปิดช่องโหว่การแอบจับเวลานี้ได้อย่างสมบูรณ์

Q8 ระบบมีมาตรการป้องกันปัญหาการข้ามสิทธิ์ดูข้อมูลของสมาชิกคนอื่น (Insecure Direct Object References - IDOR) ใน Member Portal อย่างไร?

> **เจตนาของผู้สอบ:** ตรวจสอบว่าระบบมีมาตรการ Zero-Trust ในระดับ API หรือเชื่อถือ Parameter ที่ Client ส่งมา (เช่น ?memberId=...) หรือไม่

แนวทางการตอบ:

ในช่องทาง **Member Portal** (พอร์ทัลสมาชิก) ระบบใช้มาตรการ **Server-Derived Identity (ตัวตนที่เซิร์ฟเวอร์ระบุเอง)**:

- **ไม่ยอมรับ ID จาก Client:** ใน Endpoint `/api/member-portal/*` (เช่น การดูยอดเงินคงเหลือ, การดูประวัติขายยาง, หรือการดูเงินปันผล) API จะไม่รับ `memberId` ผ่าน URL Query หรือ Request Body จากหน้าเว็บโดยเด็ดขาด
- **ดึง ID จากเซสชันที่ผ่านการยืนยันตัวตนแล้วเท่านั้น:** เซิร์ฟเวอร์จะถอดรหัส `clover_member_session` คุกกี้ที่ลงนาม HMAC ไว้แล้วดึง `memberId` จาก Payload ภายในเซสชันที่ตรวจสอบความถูกต้องแล้วเท่านั้น แล้วนำค่านั้นไปเป็นเงื่อนไข `where: { memberId: session.memberId }` ในการคิวรีฐานข้อมูล

ด้วยหลักการนี้ ต่อให้สมาชิกมีความรู้ทางเทคนิคและพยายามแก้ไขค่า HTTP Request หรือดัดแปลงสคริปต์หน้าเว็บ ก็จะไม่สามารถเข้าถึงข้อมูลของสมาชิกรายอื่นได้ เพราะเซิร์ฟเวอร์จะตอบกลับเฉพาะข้อมูลของเจ้าของเซสชันนั้นเสมอ

Q9 กลไก usingDefaultPassword ทำงานอย่างไร และเหตุใดจึงถือเป็นข้อกำหนดความปลอดภัยที่สำคัญสำหรับระบบสารสนเทศของสหกรณ์?

> **เจตนาของผู้สอบ:** วัดความเข้าใจเรื่อง Security Governance, Password Lifecycle Management และความรับผิดชอบต่อข้อมูลส่วนบุคคล (PDPA)

แนวทางการตอบ:

1. หลักการทำงาน:
เมื่อผู้ดูแลระบบ (Admin) ทำการสร้างบัญชีเจ้าหน้าที่ใหม่ในตาราง `Staff` ระบบจะสร้างรหัสผ่านเริ่มต้น (Default Password) ให้พร้อมกำหนดฟิลด์แฟล็ก `usingDefaultPassword: true` ในฐานข้อมูล เมื่อเจ้าหน้าที่คนดังกล่าวเข้าสู่ระบบสำเร็จ Middleware `accessGuard` จะตรวจพบแฟล็กนี้ และจะ **บล็อกการเข้าถึงหน้าเว็บและ API ทั้งหมดของระบบ** โดยอนุญาตเฉพาะหน้าจอเปลี่ยนรหัสผ่านและ Endpoint อัปเดตรหัสผ่านใหม่เท่านั้น เมื่อเจ้าหน้าที่ตั้งรหัสผ่านใหม่ส่วนตัวสำเร็จ ระบบจะอัปเดตแฟล็กเป็น `false` จึงจะสามารถใช้งานระบบงานทั่วไปได้

2. ความสำคัญสำหรับระบบสหกรณ์:
ระบบสหกรณ์ถือครองข้อมูลทางการเงิน ข้อมูลที่ดินสวนยาง และข้อมูลส่วนบุคคลของสมาชิกจำนวนมาก รหัสผ่านเริ่มต้นมักถูกส่งผ่านช่องทางที่ไม่ปลอดภัย (เช่น จดบนกระดาษ หรือส่งผ่านแชต) หากไม่มีกลไกบังคับเปลี่ยนรหัสผ่าน เจ้าหน้าที่อาจละเลยไม่ยอมเปลี่ยน ทำให้บุคคลอื่นสามารถสวมรอยเข้ามาทำธุรกรรมทุจริต (เช่น เบิกเงิน หรือแก้ไขสัดส่วนรายได้) กลไกนี้จึงเป็นข้อบังคับตามหลัก *Non-Repudiation (การไม่สามารถปฏิเสธความรับผิดชอบได้)* เพื่อให้มั่นใจว่าทุกการกระทำใน Audit Log เกิดจากเจ้าของบัญชีตัวจริง

Q10 ระบบมีมาตรการป้องกันการโจมตีทางเว็บที่พบบ่อย (Common Web Attacks) เช่น SQL Injection, CSRF, XSS และ Directory Traversal อย่างไรบ้าง?

> **เจตนาของผู้สอบ:** ตรวจสอบว่าระบบมีภูมิคุ้มกันความปลอดภัยรอบด้านตามมาตรฐาน OWASP Top 10 หรือไม่

แนวทางการตอบ:

ระบบมีมาตรการป้องกันแบบเชิงลึก (Defense in Depth) ครอบคลุมทุกช่องโหว่สำคัญ:

- **SQL Injection:** ป้องกันได้ 100% ผ่านการใช้ **Prisma ORM** ซึ่งประมวลผลคำสั่งฐานข้อมูลทั้งหมดด้วย *Parameterized Queries* และ Prepared Statements ข้อมูลนำเข้าจากภายนอกจะถูกส่งเป็น Parameter แยกต่างหากจากไวยากรณ์ SQL จึงไม่สามารถถูกฉีดโค้ดอันตรายเข้ามาได้
- **Cross-Site Scripting (XSS):** คุกกี้เซสชันทั้งหมดถูกกำกับด้วยแฟล็ก `HttpOnly` ทำให้ JavaScript ฝั่ง Client ไม่สามารถขโมยโทเค็นได้ และการเรนเดอร์ข้อมูลบนหน้าเว็บมีการ Escape อักขระพิเศษของ HTML เสมอ
- **Cross-Site Request Forgery (CSRF):** คุกกี้เซสชันถูกกำหนดด้วยแฟล็ก `SameSite=Lax` ซึ่งป้องกันไม่ให้เบราว์เซอร์ส่งคุกกี้ไปด้วยเมื่อมีคำขอแบบ Cross-Site ข้ามโดเมนที่ไม่ได้เกิดจากการคลิกลิงก์ปกติของผู้ใช้
- **Directory Traversal:** ใน `express.static` และการรวมหน้าด้วย `@include` มีการใช้ `path.resolve` และ `path.join` ผนวกกับการตั้งค่าไม่อนุญาตให้อ่านไฟล์ออกนอกไดเรกทอรีโครงการ

หมวดที่ 3: ความถูกต้องทางข้อมูล ธุรกรรมการเงิน และความพร้อมกัน (Data Integrity, ACID & Concurrency)

Q11 ในการบันทึกการรับซื้อน้ำยาง (Purchase) และการเบิกเงิน (Withdrawal) ระบบรับประกันคุณสมบัติ ACID และป้องกันข้อมูลคลาดเคลื่อนได้อย่างไร?

> **เจตนาของผู้สอบ:** ตรวจสอบความเข้าใจเรื่อง Transaction Management และคุณสมบัติของฐานข้อมูลระดับองค์กร (Atomicity, Consistency, Isolation, Durability)

แนวทางการตอบ:

ระบบใช้ **Interactive Transactions / Batch Transactions ของ Prisma (** `prisma.$transaction` **)** ในการควบคุมธุรกรรมการเงินทุกประเภท:

- **Atomicity (ความเป็นหนึ่งเดียว):** ในฟังก์ชัน `createPurchase` การสร้างรายการรับซื้อในตาราง `Purchase` และการบวกเงินเพิ่มใน `Member.walletBalance` ถูกผูกไว้ใน Transaction เดียวกัน หากขั้นตอนใดขั้นตอนหนึ่งล้มเหลว (เช่น เซิร์ฟเวอร์ดับ, เน็ตเวิร์กหลุด หรือติด Database Constraint) การทำงานทั้งหมดจะถูก **Rollback (ย้อนกลับ)** ทันที จะไม่มีกรณีที่สร้างรายการซื้อสำเร็จแต่เงินไม่เข้าบัญชีสมาชิก
- **Consistency (ความถูกต้องสอดคล้อง):** ข้อมูลตัวเลขเงินและเปอร์เซ็นต์ถูกกำหนดด้วยชนิดข้อมูล `Decimal(12,2)` ใน PostgreSQL ไม่มีการปัดเศษทศนิยมผิดพลาดแบบ Float
- **Isolation (ความแยกขาดจากกัน):** PostgreSQL ใช้กลไก Multi-Version Concurrency Control (MVCC) ทำให้คำขอที่ทำงานพร้อมกันไม่มองเห็นข้อมูลครึ่งๆ กลางๆ ของธุรกรรมอื่นที่ยัง Commit ไม่เสร็จ
- **Durability (ความคงทนถาวร):** เมื่อ Transaction สำเร็จ (Commit) ข้อมูลจะถูกเขียนลง Write-Ahead Logging (WAL) ของ PostgreSQL ทำให้ข้อมูลไม่สูญหายแม้ไฟดับ

Q12 สมมติว่ามีเจ้าหน้าที่ 2 คนกดอนุมัติการเบิกเงินของสมาชิกคนเดียวกันในเวลาเดียวกัน (Race Condition / Double Spending) ระบบป้องกันปัญหานี้อย่างไร?

> **เจตนาของผู้สอบ:** วัดความเข้าใจเรื่อง Concurrency Issues, Database Row-level Locking, หรือ Atomic Decrement ในระบบการเงิน

แนวทางการตอบ:

ปัญหา **Race Condition (การแย่งกันใช้ทรัพยากร)** หรือ **Double Spending** เป็นปัญหาคลาสสิกของระบบการเงิน:

- **Atomic Decrement ในระดับฐานข้อมูล:**
  ในไฟล์ `server/lib/data/withdrawals.js` คำสั่งอัปเดตยอดเงินไม่ได้ใช้วิธีคำนวณในหน่วยความจำแล้วเขียนทับแบบ `walletBalance = newBalance` แต่ใช้คำสั่งของ Prisma คือ:

  ```
  prisma.member.update({ where: { id }, data: { walletBalance: { decrement: input.amount } } })
  ```

  ซึ่งถูกแปลงเป็นคำสั่ง SQL ระดับล่าง:

  ```
  UPDATE "Member" SET "walletBalance" = "walletBalance" - $1 WHERE "id" = $2;
  ```

  ใน PostgreSQL คำสั่ง `UPDATE` จะทำการเข้าคิวและล็อกแถวข้อมูลนั้นโดยอัตโนมัติ (Implicit Row-Level Lock) คำขอที่สองจะต้องรอให้คำขอแรกทำงานเสร็จสิ้นก่อนเสมอ
- **การป้องกันยอดเงินติดลบ (Negative Balance Protection):**
  ก่อนทำการเบิก มีการตรวจสอบ `input.amount > member.walletBalance` และสามารถเสริมความปลอดภัยขั้นสูงสุดระดับฐานข้อมูลได้ด้วยการตั้งค่า Check Constraint ใน PostgreSQL เช่น `CONSTRAINT chk_wallet_non_negative CHECK (walletBalance >= 0)` ซึ่งหากคำขอที่สองมาถึงแล้วยอดเงินเหลือไม่พอ ฐานข้อมูลจะ Reject คำขอที่สองทันทีด้วยรหัสข้อผิดพลาด และ Rollback รายการเบิกนั้นออกไป

Q13 ตรรกะการคำนวณเงินและน้ำหนักในระบบรับซื้อน้ำยางและเงินปันผล ยึดหลัก Zero-Trust อย่างไร ทำไมเซิร์ฟเวอร์จึงไม่เชื่อตัวเลขที่ Client ส่งมา?

> **เจตนาของผู้สอบ:** ประเมินความเข้าใจในหลักการออกแบบ API ที่ปลอดภัย (Never Trust Client Input)

แนวทางการตอบ:

หลักการพื้นฐานที่สุดของการพัฒนาเว็บระบบการเงินคือ **"Never Trust the Client" (ห้ามเชื่อข้อมูลการคำนวณจากฝั่งเบราว์เซอร์)** เพราะเบราว์เซอร์คือสภาพแวดล้อมที่ผู้ใช้สามารถเข้าถึงและดัดแปลงได้อิสระผ่าน Developer Tools หรือเครื่องมือยิงคำขออย่าง Postman:

- **ในระบบรับซื้อ (Purchases):** หน้าเว็บอาจส่งฟิลด์ `totalAmount` หรือ `dryWeightKg` มาด้วย แต่ใน `createPurchase` (DAL) เซิร์ฟเวอร์จะ **เพิกเฉยต่อตัวเลขเหล่านั้น** และนำเฉพาะข้อมูลดิบคือ `rawWeightKg` และ `dryPercentage` มาดึงราคากลางอ้างอิงล่าสุดจากตาราง `ReferencePrice` แล้วคำนวณใหม่บนเซิร์ฟเวอร์ทั้งหมด ทำให้ผู้ใช้ไม่สามารถแอบแก้ไขราคากลางหรือยอดเงินรวมใน Payload เพื่อโกงเงินสหกรณ์ได้
- **ในระบบเงินปันผล (Dividends):** เมื่อผู้ใช้กดจ่ายปันผล หน้าเว็บจะส่งมาเพียง `buddhistYear` และ `rate` เซิร์ฟเวอร์จะทำการคิวรีตาราง `Purchase` ทั้งหมดในปีนั้นขึ้นมารวมน้ำหนักยางแห้งจริงของสมาชิกแต่ละรายด้วยตนเอง จึงมั่นใจได้ว่าเงินปันผลที่จ่ายออกไปตรงกับยอดส่งยางพาราจริง 100%

Q14 ทำไมในตาราง Withdrawal และ DividendPayment ถึงต้องมีการบันทึกฟิลด์ Snapshot เช่น balanceBefore และ balanceAfter ทั้งๆ ที่สามารถคำนวณย้อนหลังได้?

> **เจตนาของผู้สอบ:** วัดความเข้าใจเรื่อง Financial Auditing, Event Sourcing Concept, Performance of Historical Reports และ Immutable Ledgers

แนวทางการตอบ:

การจัดเก็บ **Balance Snapshot (ภาพถ่ายยอดเงิน ณ ขณะเวลาทำรายการ)** มีประโยชน์อย่างยิ่งยวดในระบบบัญชีและการเงิน:

1. **ความสามารถในการตรวจสอบทุจริตย้อนหลัง (Non-repudiation & Audit Trail):** หากเกิดข้อพิพาทว่า "ยอดเงินในบัญชีหายไปได้อย่างไร" หรือมีข้อสงสัยว่ามีใครแอบแก้ไขยอดเงินในฐานข้อมูลโดยตรง เจ้าหน้าที่สามารถตรวจสอบเส้นทางการเงินได้ทันทีว่าก่อนเบิกมีเท่าไร เบิกไปเท่าไร และหลังเบิกเหลือเท่าไร โดยตรวจสอบความสอดคล้องของสมการ `balanceAfter = balanceBefore - amount` ได้ในระดับแถวข้อมูลนั้นๆ
2. **ประสิทธิภาพในการดึงประวัติ (Query Performance):** หากไม่เก็บ Snapshot ไว้ เมื่อต้องการพิมพ์ใบเสร็จหรือแสดง Statement ย้อนหลัง เซิร์ฟเวอร์จะต้องยิงคำนวณผลรวมสะสม (Running Total Aggregation) ของทุกการซื้อและการเบิกตั้งแต่เปิดสหกรณ์ ซึ่งมีต้นทุนการประมวลผลสูงมาก (High CPU & I/O) และจะช้าลงเรื่อยๆ ตามจำนวนปีที่เพิ่มขึ้น การเก็บ Snapshot ช่วยให้อ่านข้อมูลขึ้นมาแสดงผลได้ในทันทีแบบ $O(1)$

Q15 การคำนวณและการบันทึกประวัติการจ่ายเงินปันผลประจำปี (DividendPayment) มีการป้องกันการจ่ายซ้ำในปีเดิมซ้ำซ้อนระดับ Database Constraint อย่างไร?

> **เจตนาของผู้สอบ:** ตรวจสอบว่าผู้ออกแบบระบบพึ่งพาเพียงเงื่อนไขใน Application Layer (if/else) หรือมีการใช้ Database Constraint เพื่อความปลอดภัยที่แท้จริง

แนวทางการตอบ:

ระบบมีการป้องกันแบบ 2 ชั้น (Two-Tier Defense):

- **ชั้นที่ 1: Application-Level Check (`server/lib/data/dividends.js`):** ก่อนจะเริ่มคำนวณ ฟังก์ชัน `payDividend` จะคิวรีตรวจสอบว่ามีประวัติการจ่ายเงินของปี พ.ศ. ดังกล่าวอยู่แล้วหรือไม่ หากมีจะโยน `DividendError("ปี พ.ศ. นี้ได้ทำการจ่ายเงินปันผลไปแล้ว")` ทันที
- **ชั้นที่ 2: Database-Level Constraint (`prisma/schema.prisma`):** ในโมเดล `DividendPayment` มีการกำหนดเงื่อนไขสำคัญไว้คือ:

  ```
  @@unique([memberId, buddhistYear])
  ```

  เงื่อนไขนี้บังคับให้ในตาราง PostgreSQL สมาชิก 1 คนจะมีเรคคอร์ดการจ่ายปันผลในปี พ.ศ. เดียวกันได้ไม่เกิน 1 แถวอย่างเด็ดขาด หากเกิดเหตุการณ์ที่มีผู้ดูแลระบบ 2 คนกดปุ่ม "จ่ายปันผล" พร้อมกันในมิลลิวินาทีเดียวกัน จนผ่านการตรวจสอบในชั้นที่ 1 มาได้ทั้งคู่ ฐานข้อมูลจะปฏิเสธคำขอที่สองทันทีด้วยข้อผิดพลาด **Unique Constraint Violation (Prisma Error P2002)** ทำให้ไม่มีทางที่เงินปันผลจะถูกบวกเบิ้ลเข้ากระเป๋าเงินสมาชิกสองรอบได้โดยเด็ดขาด

หมวดที่ 4: การทำงานกับฐานข้อมูลและ Prisma ORM (Database Design & ORM Optimization)

Q16 อธิบายความสัมพันธ์ระหว่าง Member, Employee และ MePair ใน Prisma Schema ทำไมจึงต้องมีโมเดล MePair คั่นกลาง?

> **เจตนาของผู้สอบ:** ทดสอบความรู้เรื่องการออกแบบฐานข้อมูลเชิงสัมพันธ์ การจัดการ Many-to-Many Relationship และการจัดเก็บ Relationship Attributes

แนวทางการตอบ:

ในความเป็นจริงของธุรกิจสวนยางพารา ความสัมพันธ์ระหว่าง **สมาชิกเจ้าของสวน (Member)** และ **คนกรีดยาง (Employee)** เป็นความสัมพันธ์แบบ **Many-to-Many (กลุ่มต่อกลุ่ม)**:

- สมาชิก 1 คน อาจมีสวนยางหลายแปลง และจ้างคนกรีดยางได้หลายคน
- คนกรีดยาง 1 คน อาจรับจ้างกรีดยางให้กับสมาชิกเจ้าของสวนได้มากกว่า 1 คนในรอบสัปดาห์หรือช่วงเวลาที่ต่างกัน

โมเดล `MePair` ทำหน้าที่เป็น **Junction Table (ตารางเชื่อมโยง)** ที่ยกระดับขึ้นเป็น Entity ที่เก็บ *Relationship Attributes (คุณลักษณะของความสัมพันธ์)* ที่สำคัญ ได้แก่:

1. **สัดส่วนแบ่งรายได้ (`memberShare`, `employeeShare`):** เช่น สมาชิก 60% / คนกรีด 40% ซึ่งแต่ละคู่สัญญาอาจตกลงสัดส่วนไม่เท่ากัน จึงไม่สามารถเก็บไว้ในตาราง Member หรือ Employee โดยตรงได้
2. **ช่วงเวลาของสัญญา (`contractStartDate`, `contractEndDate`):** เพื่อบันทึกประวัติการเริ่มและสิ้นสุดสัญญา
3. **ไฟล์เอกสารแนบสัญญา (`contractFileUrl`):** บันทึกหลักฐานเอกสารสัญญาจ้างจริง

Q17 ในการคิวรีข้อมูลที่มีความสัมพันธ์หลายชั้นใน Prisma ORM มีการป้องกันปัญหา N+1 Query Problem อย่างไร?

> **เจตนาของผู้สอบ:** ประเมินทักษะการเพิ่มประสิทธิภาพการคิวรีฐานข้อมูล (Query Optimization) และการเข้าใจกลไกของ ORM

แนวทางการตอบ:

ปัญหา N+1 Query Problem: เกิดขึ้นเมื่อเราคิวรีข้อมูลแม่ 1 ครั้ง (ได้ผลลัพธ์ N แถว) แล้วเขียนลูปวนคิวรีข้อมูลลูกทีละแถว ทำให้เกิดคำสั่งคิวรีไปยังฐานข้อมูลรวม $1 + N$ ครั้ง ซึ่งทำให้ฐานข้อมูลทำงานหนักและช้าลงอย่างมาก

แนวทางแก้ไขใน Clover Project:

- **การใช้ `include` หรือ `select` ของ Prisma:** ในไฟล์อย่าง `server/lib/data/contracts.js` เราใช้:

  ```
  const pairs = await prisma.mePair.findMany({ include: { member: true, employee: true } });
  ```

  Prisma จะแปลงคำสั่งนี้เป็นการคิวรีที่เหมาะสม โดยคิวรีข้อมูลหลักแล้วใช้ `IN (...)` เพื่อดึงข้อมูลความสัมพันธ์ทั้งหมดในคราวเดียว หรือทำ `JOIN` ที่ระดับฐานข้อมูล ทำให้เหลือคำสั่งคิวรีเพียง 1 หรือ 2 คำสั่งเท่านั้น
- **การใช้ `Promise.all` สำหรับข้อมูลที่ไม่เกี่ยวข้องกันโดยตรง:** เช่น ใน `getDividendData()` ข้อมูลสมาชิก ข้อมูลการซื้อ และประวัติการจ่ายปันผล จะถูกคิวรีแบบขนาน (Parallel Execution) ผ่าน `Promise.all([prisma.member..., prisma.purchase..., prisma.dividendPayment...])` ทำให้ใช้เวลาเท่ากับคิวรีที่ช้าที่สุดเพียงตัวเดียว แทนที่จะรอต่อคิวกันแบบ Sequential

Q18 มีการออกแบบ Database Index อย่างไรใน schema.prisma เพื่อรองรับการค้นหาและสรุปยอดที่มีข้อมูลปริมาณมาก?

> **เจตนาของผู้สอบ:** ตรวจสอบความเข้าใจเรื่อง Indexing Strategies, B-Tree Indexes และ Query Execution Plan

แนวทางการตอบ:

ใน `prisma/schema.prisma` มีการวิเคราะห์พฤติกรรมการใช้งานจริง (Query Workload Analysis) และทำ Index ในจุดที่มีการค้นหาบ่อย:

- **Foreign Key Indexing:** มีการเพิ่ม `@@index([memberId])` ในตาราง `Purchase`, `Withdrawal`, `DividendPayment` และ `MePair` ช่วยให้การค้นหาประวัติการทำธุรกรรมของสมาชิกรายคนทำได้อย่างรวดเร็วในระดับ $O(\log N)$
- **Time-Series & Range Query Indexing:** มีการเพิ่ม `@@index([recordDate])` ในตาราง `Purchase` และ `@@index([actionTime])` ในตาราง `ActivityLog` ทำให้การดึงข้อมูลเพื่อสรุปยอดขายรายวัน รายเดือน หรือการกรองตามช่วงเวลา (Date Range Filtering) สามารถใช้ Index Scan ได้อย่างมีประสิทธิภาพโดยไม่ต้องทำ Full Table Scan
- **Composite Indexing:** ในตาราง `ActivityLog` มีการทำ Index แบบคู่คือ `@@index([targetType, targetId])` รองรับการค้นหาประวัติการเปลี่ยนแปลงที่เกิดขึ้นกับเอนทิตีเฉพาะตัวได้อย่างรวดเร็ว

Q19 หากในอนาคตฐานข้อมูลมีรายการรับซื้อ (Purchase) หลายล้านแถว Backend มีกลยุทธ์การทำ Pagination หรือ Data Aggregation อย่างไร?

> **เจตนาของผู้สอบ:** ประเมินวิสัยทัศน์ด้าน Scalability, Big Data Handling และการออกแบบสถาปัตยกรรมสำหรับอนาคต

แนวทางการตอบ:

หากระบบเติบโตจนมีข้อมูลหลายล้านรายการ จะประยุกต์ใช้ 3 กลยุทธ์หลัก:

1. **เปลี่ยนจาก Offset-based เป็น Keyset Pagination (Cursor Pagination):** การใช้ `skip: 1000000, take: 50` ใน SQL แบบเดิมจะช้าลงเรื่อยๆ เพราะฐานข้อมูลต้องสแกนข้ามล้านแถวแรก การเปลี่ยนมาใช้ Cursor Pagination ผ่าน Prisma เช่น `cursor: { id: lastSeenId }, take: 50` ร่วมกับ Index จะทำให้เวลาในการดึงข้อมูลหน้าถัดไปคงที่เท่ากันทุกหน้า ไม่ว่าข้อมูลจะอยู่ลึกแค่ไหน
2. **การใช้ Database Aggregation Function:** แทนที่จะดึงรายการทั้งหมดมาคำนวณผลรวมใน JavaScript ให้ส่งคำสั่ง `prisma.purchase.aggregate({ _sum: { dryWeightKg: true, totalAmount: true }, where: { ... } })` เพื่อให้ PostgreSQL ประมวลผลผลรวมในระดับ Storage Engine ซึ่งเร็วและกิน Memory น้อยกว่ามาก
3. **Table Partitioning:** แบ่งตาราง `Purchase` ในระดับ PostgreSQL ตามปี พ.ศ. (Partition by Range บนคอลัมน์ `recordDate`) เพื่อให้การสืบค้นข้อมูลของปีปัจจุบันไม่ต้องไปยุ่งเกี่ยวกับข้อมูลย้อนหลังของปีก่อนๆ

Q20 การลบข้อมูล (Delete) ในระบบนี้ใช้แบบ Soft Delete หรือ Hard Delete และมีผลกระทบต่อ Referential Integrity อย่างไร?

> **เจตนาของผู้สอบ:** ตรวจสอบความเข้าใจเรื่อง Data Retention, Referential Integrity และกฎหมายข้อมูลทางบัญชี

แนวทางการตอบ:

ในระบบบัญชีการเงิน **ห้ามใช้ Hard Delete (การลบแถวทิ้งออกจากฐานข้อมูลอย่างถาวร) กับข้อมูลหลักโดยเด็ดขาด**:

- **การใช้ Soft Delete / Status Deactivation:** ในโมเดล `Member` และ `Employee` มีฟิลด์ `status` ที่มีค่าเป็น `Active` หรือ `Inactive` เมื่อสมาชิกหรือคนกรีดยางต้องการยกเลิกการใช้งาน ระบบจะอัปเดตสถานะเป็น `Inactive` เท่านั้น
- **การรักษาสภาพความสมบูรณ์ของความสัมพันธ์ (Referential Integrity):** หากเราทำการ Hard Delete สมาชิกที่เคยส่งน้ำยางไปแล้ว รายการในตาราง `Purchase`, `Withdrawal` และ `DividendPayment` ที่อ้างอิง Foreign Key `memberId` จะเกิดสภาวะกำพร้า (Orphaned Records) ทำให้รายงานสรุปยอดบัญชีของสหกรณ์คลาดเคลื่อนทันที การใช้ Soft Delete ทำให้ข้อมูลการเงินในอดีตยังคงสามารถเชื่อมโยงและตรวจสอบย้อนหลังได้ครบถ้วนสมบูรณ์ 100%

หมวดที่ 5: ประสิทธิภาพ ความเสถียร และการบำรุงรักษาในระยะยาว (Performance & Robustness)

Q21 ระบบจัดการ Error Handling อย่างไรเพื่อไม่ให้ Node.js Process เกิด Unhandled Promise Rejection จนเซิร์ฟเวอร์ Crash?

> **เจตนาของผู้สอบ:** วัดความเข้าใจในระบบ Async/Await Lifecycle, Error Boundaries และ High Availability

แนวทางการตอบ:

ระบบใช้โครงสร้างการจัดการข้อผิดพลาดแบบหลายชั้น (Multi-Tier Error Handling Architecture):

- **Custom Error Classes (`server/lib/api-error.js`):** สร้างคลาสมาตรฐานอย่าง `ApiError`, `SellerLookupError`, `WithdrawalError`, และ `DividendError` ทำให้สามารถแยกแยะได้ชัดเจนระหว่าง *Operational Errors* (ข้อผิดพลาดที่คาดการณ์ได้ เช่น เงินไม่พอ, รหัสไม่พบ) กับ *Programmer/System Bugs* (เช่น ข้อผิดพลาดฐานข้อมูล)
- **Express Async Error Propagation:** ด้วยการอัปเกรดมาใช้ **Express 5.2.1** ซึ่งมีจุดเด่นสำคัญที่สุดคือ **รองรับ Native Async/Await Error Handling ในตัว** ใน Express 4 หากฟังก์ชัน `async` โยน Error โดยไม่มี try/catch เซิร์ฟเวอร์จะเกิด *UnhandledPromiseRejection* และล่มได้ แต่ใน Express 5 ระบบจะส่ง Error ของ Promise ที่ถูก Reject ไปยัง Error-handling Middleware ตัวสุดท้ายโดยอัตโนมัติ
- **Centralized Error Middleware (`server/server.js`):** ดักจับ Error สุดท้ายที่บรรทัด 56-64 ทำการบันทึก `console.error(error)` และส่ง HTTP Status พร้อมข้อความ JSON ที่เหมาะสมกลับไปให้ไคลเอนต์เสมอ ทำให้โปรเซสของเซิร์ฟเวอร์ยังคงทำงานต่อไปได้อย่างเสถียร

Q22 กลไก Activity Log มีการบันทึกข้อมูลอะไรบ้าง และหากการบันทึก Log ล้มเหลว จะส่งผลให้ Business Transaction หลักล้มเหลวไปด้วยหรือไม่?

> **เจตนาของผู้สอบ:** ตรวจสอบความเข้าใจเรื่อง Audit Logging Policy และ Decoupling ระหว่าง Core Business Transaction กับ Observability/Auditing

แนวทางการตอบ:

1. ข้อมูลที่บันทึกใน `ActivityLog`:
บันทึกข้อมูลสำคัญ ได้แก่ รหัสเจ้าหน้าที่ผู้กระทำ (`staffId`), ประเภทการกระทำ (`ActionType` เช่น `CREATE_PURCHASE`, `LOGIN_FAILED`), ประเภทเป้าหมาย (`TargetType`), รหัสอ้างอิงเป้าหมาย (`targetId`), สถานะสำเร็จหรือล้มเหลว (`ActionStatus`), คำอธิบายรายละเอียดเพิ่มเติม (`description`) และเวลาที่เกิดเหตุการณ์ (`actionTime`)

2. นโยบายเมื่อการบันทึก Log เกิดข้อผิดพลาด:
ในเชิงสถาปัตยกรรม การบันทึก Activity Log ถูกออกแบบให้ทำงานแบบ **Non-blocking / Isolated Execution** โดยการทำงานของ Log จะถูกครอบด้วย `try/catch` แยกต่างหาก หากเกิดเหตุสุดวิสัยที่ทำให้การเขียน Log ล้มเหลว (เช่น Disk เต็มเฉพาะส่วน หรือติด Index ล็อก) ระบบจะทำการพ่น Error ออกทาง System Console แต่จะไม่ขัดขวางหรือย้อนกลับ (Rollback) ธุรกรรมหลักที่สำเร็จไปแล้ว (เช่น การส่งน้ำยางหรือการเบิกเงิน) เพื่อให้การดำเนินงานทางธุรกิจของสหกรณ์สามารถขับเคลื่อนต่อไปได้โดยไม่หยุดชะงัก

Q23 การส่งออกไฟล์สรุปยอดซื้อขายเป็น Excel ด้วย ExcelJS มีการจัดการเรื่อง Memory Usage อย่างไร เมื่อต้องจัดการข้อมูลขนาดใหญ่?

> **เจตนาของผู้สอบ:** ประเมินทักษะการจัดการหน่วยความจำ (Memory Management) และ Stream Processing เมื่อต้องประมวลผลไฟล์ขนาดใหญ่

แนวทางการตอบ:

ใน `server/lib/export/purchase-summary-workbook.js` มีการจัดการดังนี้:

- **การดึงข้อมูลเฉพาะฟิลด์ที่จำเป็น (Selective Projection):** ก่อนนำข้อมูลมาสร้าง Excel มีการใช้ Prisma `select` ดึงเฉพาะคอลัมน์ที่ต้องแสดงในรายงานเท่านั้น เช่น รหัส, วันที่, น้ำหนัก, ยอดเงิน ตัดฟิลด์ขนาดใหญ่อย่าง Base64 Photo หรือข้อมูลที่อยู่ที่ไม่เกี่ยวข้องออก ช่วยลด Memory Footprint ของ JavaScript V8 Heap ลงได้กว่า 70%
- **การสตรีมและส่งออกเป็น Buffer:** เวิร์กบุ๊กถูกคอมไพล์เป็นไบนารี Buffer และส่งผ่าน `res.setHeader("Content-Type", "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")` แล้วเขียนลงใน HTTP Response Stream ทันที
- **แนวทางขยายต่อยอดสำหรับข้อมูลระดับล้านแถว:** ExcelJS มีฟีเจอร์ `ExcelJS.stream.xlsx.WorkbookWriter` ซึ่งรองรับการเขียนข้อมูลทีละแถวลงดิสก์/เน็ตเวิร์กสตรีมทันที (Row-by-Row Streaming) โดยไม่จำเป็นต้องเก็บทั้ง Spreadsheet ไว้ในหน่วยความจำ RAM ทำให้สามารถสร้างไฟล์ Excel ขนาดหลายร้อยเมกะไบต์ได้โดยใช้ RAM เพียงไม่กี่สิบเมกะไบต์

Q24 ระบบจัดการเรื่อง Timezone และวันที่ระหว่าง UTC ใน PostgreSQL กับ วันที่ปี พ.ศ. (Thai Buddhist Era) อย่างไรเพื่อไม่ให้เกิดข้อผิดพลาดในการตัดรอบบัญชี?

> **เจตนาของผู้สอบ:** ตรวจสอบความเข้าใจเรื่อง Timezone Pitfalls, UTC Storage และการจัดการปฏิทินท้องถิ่น

แนวทางการตอบ:

ปัญหาเรื่อง Timezone เป็นสาเหตุอันดับต้นๆ ที่ทำให้ระบบบัญชีตัดยอดวันผิดพลาด (เช่น ยอดขายตอนเช้าของวันที่ 15 หลุดไปเป็นของวันที่ 14):

- **Single Source of Truth in UTC:** ในฐานข้อมูล PostgreSQL วันที่ทั้งหมดถูกจัดเก็บตามมาตรฐานสากลคือ **UTC (Coordinated Universal Time)** ฟิลด์ `recordDate` จะถูก normalize ให้เป็นเที่ยงคืนของวันทำการนั้นๆ ในรูปแบบ UTC เสมอ
- **Deterministic Year Conversion (`dividends.js`):** ในการตัดรอบปี พ.ศ. ของเงินปันผล ระบบมีฟังก์ชันเฉพาะ:

```
function buddhistYearOf(date) { return date.getUTCFullYear() + 543; }
```

  การเจาะจงใช้ `date.getUTCFullYear()` แทน `date.getFullYear()` เป็นสิ่งสำคัญมาก เพราะจะอิงตามปีสากลของวันที่ถูกบันทึกจริง ไม่ขึ้นอยู่กับ Timezone ของเครื่องเซิร์ฟเวอร์ที่กำลังรันโค้ดอยู่ ทำให้การตัดรอบปีปันผลมีความแน่นอน 100% (Deterministic)
- **Presentation Layer Formatting (`shared/format.js`):** การแปลงวันที่ให้เป็นภาษาไทยและปี พ.ศ. สวยงามสำหรับผู้ใช้ (เช่น "15 ตุลาคม 2569") จะเกิดขึ้นเฉพาะในเลเยอร์การแสดงผลเท่านั้น โดยใช้ `Intl.DateTimeFormat('th-TH', { timeZone: 'Asia/Bangkok' })`

Q25 หากต้องการนำระบบ Backend นี้ขึ้น Production ในสภาพแวดล้อมจริง (เช่น Docker, Cloud VM หรือ Kubernetes) ต้องมีขั้นตอนและแนวทางการปรับปรุงความปลอดภัย (Production Hardening) อะไรเพิ่มเติมบ้าง?

> **เจตนาของผู้สอบ:** ประเมินความพร้อมสู่การใช้งานจริง (Production Readiness, DevOps & Infrastructure Security)

แนวทางการตอบ:

ขั้นตอนและแนวปฏิบัติในการนำระบบขึ้นสู่สภาวะ Production จริงตามมาตรฐานสากล:

1. **Containerization ด้วย Docker:** สร้าง `Dockerfile` แบบ Multi-stage Build โดยใช้ Image พื้นฐานที่เป็น `node:20-alpine` (ขนาดเล็กและปลอดภัย) และรันโปรเซสด้วยสิทธิ์ผู้ใช้ทั่วไปที่ไม่ใช่ Root (`USER node`)
2. **HTTPS & Cookie Security Flags:** ติดตั้ง SSL/TLS Certificate (เช่น Let's Encrypt) และเมื่อรันในโหมด `NODE_ENV=production` คุกกี้เซสชันจะเปิดใช้งานแฟล็ก `Secure: true` โดยอัตโนมัติ เพื่อบังคับให้ส่งคุกกี้ผ่านช่องทางที่เข้ารหัส HTTPS เท่านั้น
3. **Security Headers & Rate Limiting:** ติดตั้ง `helmet` เพื่อเพิ่ม Security Headers เช่น Content-Security-Policy (CSP), HSTS และ X-Content-Type-Options พร้อมติดตั้ง `express-rate-limit` ในเส้นทาง `/api/auth/login` เพื่อป้องกันการโจมตีแบบ Brute-Force คาดเดารหัสผ่าน
4. **Process Management & Monitoring:** ใช้ **PM2** ในโหมด Cluster หรือรันบน **Kubernetes Pods** ร่วมกับ Health Check Endpoints (`/healthz`) เพื่อให้เซิร์ฟเวอร์รีสตาร์ตตัวเองอัตโนมัติหากเกิดปัญหาวิกฤต และทำ Zero-Downtime Deployment
5. **Database Backup & Disaster Recovery:** ตั้งค่า Automated Daily Backup และ Point-in-Time Recovery (PITR) ของ PostgreSQL บนคลาวด์ เพื่อรับประกันความปลอดภัยของข้อมูลธุรกรรมการเงินของสหกรณ์